

Project: Crypto Execution Optimizer

What it predicts: This system predicts the optimal execution strategy—Aggressive, Balanced, or Passive—for deploying "late capital" into an active cryptocurrency momentum trend to minimize implementation shortfall over a 72-hour window.

Data source: Data is ingested through an automated pipeline utilizing an AWS Lambda function triggered by a one-minute cron job. This process fetches real-time OHLCV data from the Binance REST API and persists the raw data into an Amazon S3 bucket. The ingestion frequency was mathematically optimized to respect API rate limits while ensuring a persistent, minute-level market archive. While this specific project utilizes the most recently 3 months approximately 130,000 rows of BTC/USDC data, the S3-based data lake captures a broader set of trading metrics, providing substantial flexibility for future research and high-resolution backtesting.

Pipeline Architecture:

1. Data is ingested from AWS Lambda into Amazon S3 (Landing Zone).
2. Within Databricks, we employ a **Delta Lakehouse** Medallion Architecture:
 1. Bronze Layer: Houses the raw data ingested from S3, ensuring a single source of truth.
 2. Silver Layer: Cleansed and standardized data where timestamps are normalized and schema enforcement is applied using PySpark and Delta format.
 3. Gold Layer: Aggregated, high-order features optimized for Machine Learning models and analytics.
3. Utilizes FLAML for automated model training and MLflow for experiment tracking and model registry.
4. The final model is deployed via Databricks Serverless Model Serving, exposing a REST API that powers a publicly hosted Streamlit web application.

[Binance API] → [AWS Lambda] → [Amazon S3] → [Databricks Spark] → [MLflow] → [Model Serving API] → [Streamlit UI]

Model approach: The system establishes a robust MLOps pipeline using a FLAML-trained Extra Trees Classifier with a strict Out-of-Time chronological split to prevent data leakage. Using features like moving average bias, volatility, and momentum duration, the model achieved a baseline macro F1-score of 0.4009. This realistically reflects crypto market noise under zero-leakage conditions, successfully validating the automated pipeline. Performance was constrained by a 5-minute compute budget (preventing convergence) and marginal target strategy cost differences. Future optimizations will increase compute time, integrate broader macro-signals, and apply label thresholding to minimize noise.

What I learned: The challenge was bridging model training and real-time inference, specifically navigating MLflow's strict schema enforcement and parsing complex JSON probability matrices. Overcoming this highlighted Databricks' strength as a unified platform for ML execution, metric logging, and version control. Given more time, I would scale the data architecture. First, I would implement Databricks Auto Loader to evaluate continuous, low-latency ingestion for real-time analysis. Second, I would ingest high-volume micro-structure data—like Level 2 order books and tick-level trades—to significantly enhance the model's predictive edge.